

Given a bank customer, build a neural network-based classifier that can determine whether they will leave or not in the next 6 months.

Dataset Description: The case study is from an open-source dataset from Kaggle. The dataset contains 10,000 sample points with 14 distinct features such as CustomerId, CreditScore, Geography, Gender, Age, Tenure, Balance, etc. Link to the Kaggle project: <https://www.kaggle.com/barelydedicated/bank-customer-churn-modeling> Perform following steps:

Read the dataset.

Distinguish the feature and target set and divide the data set into training and test sets.

Normalize the train and test data.

Initialize and build the model. Identify the points of improvement and implement the same.

Print the accuracy score and confusion matrix.

```
from google.colab import drive
```

```
drive.mount("/content/drive")
```

➞ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive")

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv("/content/drive/MyDrive/Churn_Modelling.csv")
df.head()
```

➞

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfPr
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	
3	4	15701354	Boni	699	France	Female	39	1	0.00	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	

```
df.shape
```

➞ (10000, 14)

```
df.describe()
```



	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000

```
df.isnull()
```



	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumO
0	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...
9995	False	False	False	False	False	False	False	False	False	False
9996	False	False	False	False	False	False	False	False	False	False
9997	False	False	False	False	False	False	False	False	False	False
9998	False	False	False	False	False	False	False	False	False	False
9999	False	False	False	False	False	False	False	False	False	False

10000 rows × 14 columns

```
df.isnull().sum()
```



RowNumber	0
CustomerId	0
Surname	0
CreditScore	0
Geography	0
Gender	0
Age	0
Tenure	0
Balance	0
NumOfProducts	0
HasCrCard	0
IsActiveMember	0
EstimatedSalary	0
Exited	0
dtype: int64	

```
df.info()
```

```
↗ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   RowNumber              10000 non-null  int64
1   CustomerId             10000 non-null  int64
2   Surname                10000 non-null  object
3   CreditScore             10000 non-null  int64
4   Geography               10000 non-null  object
5   Gender                 10000 non-null  object
6   Age                    10000 non-null  int64
7   Tenure                 10000 non-null  int64
8   Balance                10000 non-null  float64
9   NumOfProducts          10000 non-null  int64
10  HasCrCard              10000 non-null  int64
11  IsActiveMember         10000 non-null  int64
12  EstimatedSalary        10000 non-null  float64
13  Exited                 10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

```
df.dtypes
```

```
↗ RowNumber          int64
   CustomerId        int64
   Surname            object
   CreditScore        int64
   Geography          object
   Gender             object
   Age                int64
   Tenure             int64
   Balance            float64
   NumOfProducts      int64
   HasCrCard          int64
   IsActiveMember     int64
   EstimatedSalary    float64
   Exited             int64
dtype: object
```

```
df.columns
```

```
↗ Index(['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography',
        'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
        'IsActiveMember', 'EstimatedSalary', 'Exited'],
        dtype='object')
```

```
df.head()
```

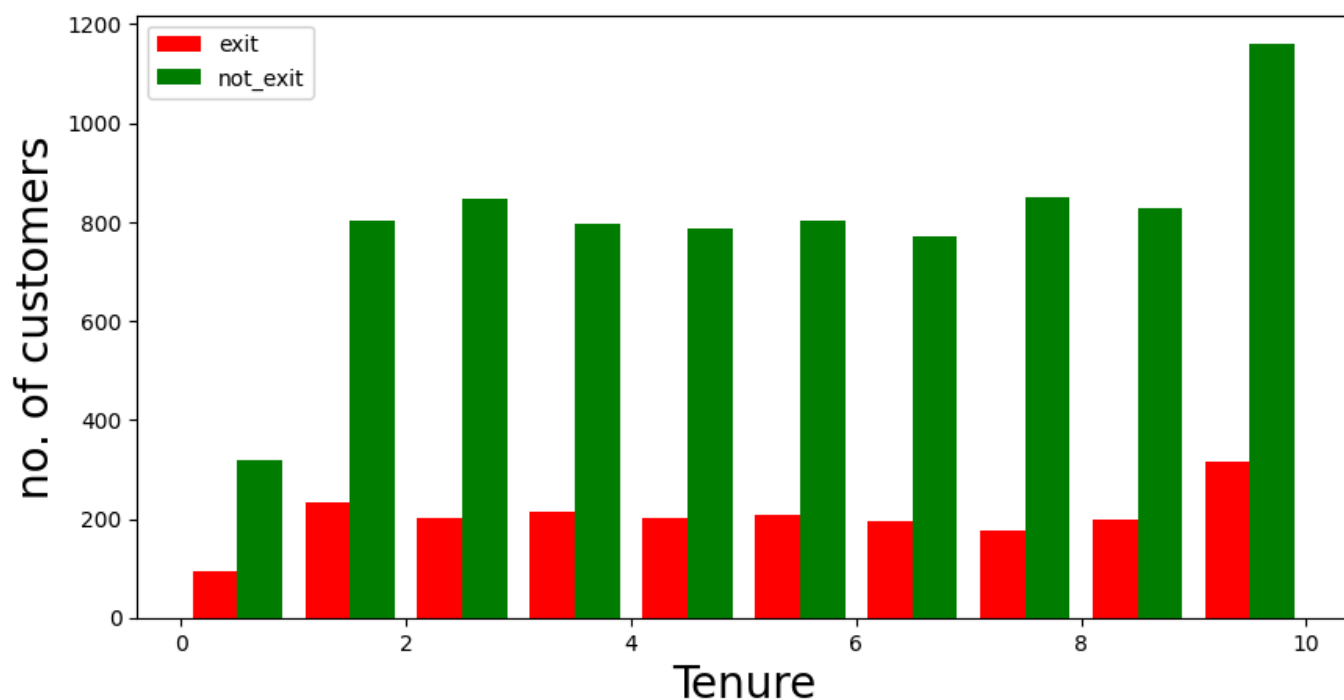


	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfPr
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	
3	4	15701354	Boni	699	France	Female	39	1	0.00	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	

```
def visualization(x, y, xlabel):  
    plt.figure(figsize=(10,5))  
    plt.hist([x, y],color=['red','green'],label=['exit','not_exit'])  
    plt.xlabel(xlabel,fontsize=20)  
    plt.ylabel('no. of customers',fontsize=20)  
    plt.legend()
```

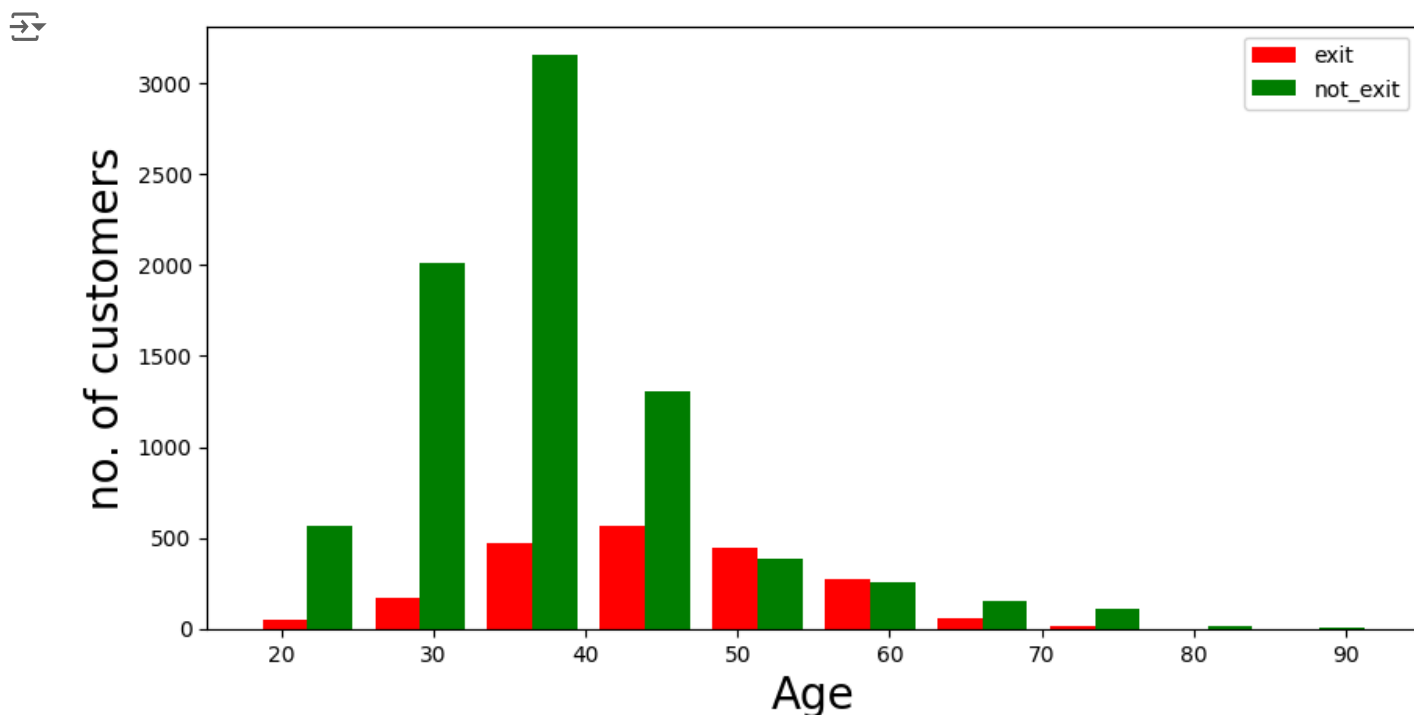
```
df_churn_exited=df[df['Exited']==1]['Tenure']  
df_churn_not_exited=df[df['Exited']==0]['Tenure']
```

```
visualization(df_churn_exited,df_churn_not_exited,'Tenure')
```



```
df_churn_exited2=df[df['Exited']==1]['Age']  
df_churn_not_exited2=df[df['Exited']==0]['Age']
```

```
visualization(df_churn_exited2,df_churn_not_exited2,'Age')
```



```
X = df[['CreditScore','Gender','Age','Tenure','Balance','NumOfProducts','HasCrCard','IsActiveMembe
states = pd.get_dummies(df['Geography'],drop_first = True)
gender = pd.get_dummies(df['Gender'],drop_first = True)
```

```
df = pd.concat([df,gender,states], axis = 1)
```

```
df.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfPr
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	
3	4	15701354	Boni	699	France	Female	39	1	0.00	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	

```
X=df[['CreditScore','Age','Tenure','Balance','NumOfProducts','HasCrCard','IsActiveMember','Estimat
```

```
y=df['Exited']
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.30)
```

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

X_train

```
→ array([[ -0.94085634,  2.93756407,  1.37497454, ..., -1.09507222,
          -0.57757021, -0.56503205],
        [ 0.95801994, -0.66616072,  0.33624998, ..., -1.09507222,
          1.73139123, -0.56503205],
        [-0.34229751,  0.09251818, -0.00999154, ...,  0.91318178,
          -0.57757021, -0.56503205],
        ...,
        [ 0.51426081,  0.47185763, -1.0487161 , ..., -1.09507222,
          -0.57757021, -0.56503205],
        [ 1.22633941, -1.33000476,  1.72121606, ..., -1.09507222,
          -0.57757021, -0.56503205],
        [ 2.06225777,  1.51504113, -1.74119915, ...,  0.91318178,
          1.73139123, -0.56503205]])
```

X_test

```
→ array([[ 0.65874052, -1.04550017,  0.6824915 , ...,  0.91318178,
          1.73139123, -0.56503205],
        [ 0.89610006,  0.37702277,  1.02873302, ...,  0.91318178,
          1.73139123, -0.56503205],
        [-0.88925644,  1.3253714 , -1.0487161 , ..., -1.09507222,
          -0.57757021, -0.56503205],
        ...,
        [-0.15653788,  0.85119709, -0.35623306, ..., -1.09507222,
          1.73139123, -0.56503205],
        [ 1.11281963,  1.60987599, -1.39495763, ...,  0.91318178,
          -0.57757021,  1.76981112],
        [-1.48781526, -0.476491 ,  0.6824915 , ...,  0.91318178,
          -0.57757021, -0.56503205]])
```

```
import keras
```

```
from keras.models import Sequential
from keras.layers import Dense
```

```
classifier = Sequential()
```

```
classifier.add(Dense(activation = "relu",input_dim = 11,units = 6,kernel_initializer = "uniform"))
```

```
→ /usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
classifier.add(Dense(activation = "relu",units = 6,kernel_initializer = "uniform"))
```

```
classifier.add(Dense(activation = "sigmoid",units = 1,kernel_initializer = "uniform"))
```

```
classifier.compile(optimizer="adam",loss = 'binary_crossentropy',metrics = ['accuracy'])
```

```
classifier.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6)	72
dense_1 (Dense)	(None, 6)	42
dense_2 (Dense)	(None, 6)	42
dense_3 (Dense)	(None, 1)	7

Total params: 163 (652.00 B)
Trainable params: 163 (652.00 B)
Non-trainable params: 0 (0.00 B)

classifier.fit(X_train,y_train,batch_size=10,epochs=50)

700/700 2s 2ms/step - accuracy: 0.8435 - loss: 0.3957
Epoch 23/50

```
Epoch 45/50
700/700 ————— 1s 2ms/step - accuracy: 0.8403 - loss: 0.3995
Epoch 46/50
700/700 ————— 1s 2ms/step - accuracy: 0.8408 - loss: 0.3894
Epoch 47/50
700/700 ————— 1s 2ms/step - accuracy: 0.8428 - loss: 0.4008
Epoch 48/50
700/700 ————— 1s 1ms/step - accuracy: 0.8350 - loss: 0.4024
Epoch 49/50
700/700 ————— 1s 1ms/step - accuracy: 0.8430 - loss: 0.3901
Epoch 50/50
700/700 ————— 1s 2ms/step - accuracy: 0.8429 - loss: 0.3948
<keras.src.callbacks.history.History at 0x7dcecf98d810>
```

```
y_pred = classifier.predict(X_test)
y_pred = (y_pred > 0.5)
```

```
⇒ 94/94 ————— 0s 2ms/step
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
cm
```

```
⇒ array([[2312,  62],
        [ 458, 168]])
```

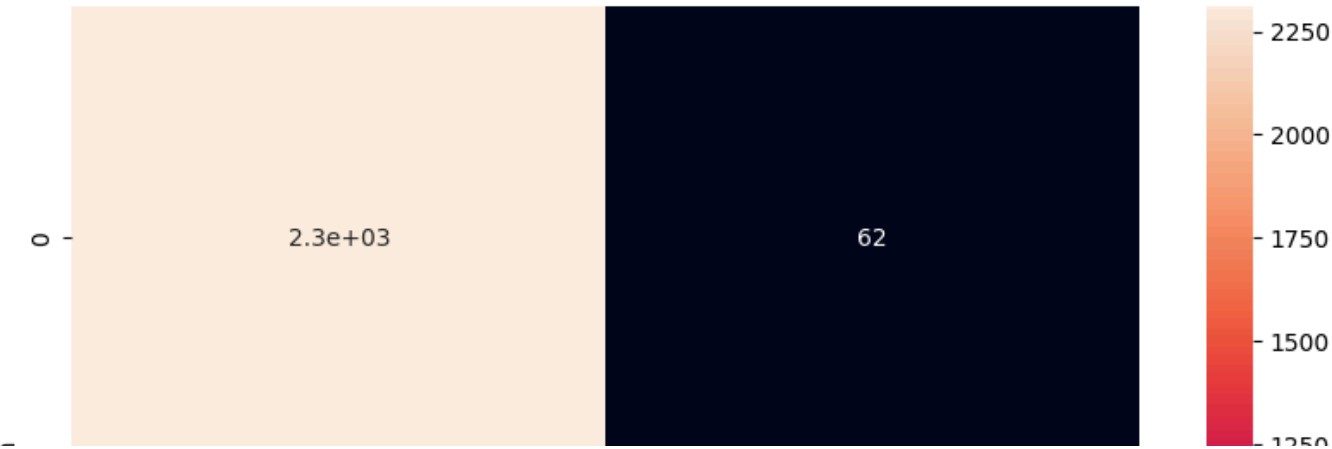
```
accuracy = accuracy_score(y_test, y_pred)
```

```
accuracy
```

```
⇒ 0.8266666666666667
```

```
plt.figure(figsize = (10,7))
sns.heatmap(cm, annot = True)
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

```
Text(95.72222222222221, 0.5, 'Truth')
```



```
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.83	0.97	0.90	2374
1	0.73	0.27	0.39	626
accuracy			0.83	3000
macro avg	0.78	0.62	0.65	3000
weighted avg	0.81	0.83	0.79	3000

